

题 目:	面向城市智慧停车的 DHPA 分解增强预测模型与 协同服务平台
姓 名:	刘福涛
学 号:	20231120137
专 业:	计算机科学与技术
日 期:	2026 年 6 月 16 日

一、背景与意义

随着城市机动车保有量持续增长，停车供需矛盾已经成为影响城市交通运行效率和居民出行体验的重要问题^[2]。停车难不仅表现为车位资源紧张，更表现为停车信息不透明、停车资源配置不均衡以及寻找车位过程中的重复绕行和时间浪费^[2]。近年来，我国围绕智慧交通与智慧停车持续推进标准化与数字化建设，2024 年发布实施的《智慧城市智慧停车第 2 部分：数据要求》从数据层面对智慧停车系统建设提出规范^[2]，2024 年发布实施的《智慧城市基础设施智慧交通中城市停车位匹配实施指南》进一步体现了城市停车资源智能匹配和精细治理的发展方向^[2]；交通运输部也在推进公路数字化转型的政策中明确提出预约停车等一体化出行服务模式^[2]。这表明，面向实时感知、智能调度和精准服务的停车预测技术，已经具有明确的政策背景和现实需求。

从研究现状看，停车位可用性预测已经由早期的统计模型、浅层机器学习方法，逐步发展到以深度学习、图神经网络和 Transformer 为代表的时空预测方法^[2]。DeepPA 基于新加坡大规模停车数据集 SINPA，提出了可在数千个停车场场景下进行联合预测的数据驱动框架，并尝试在保证可计算性的前提下建模复杂空间依赖^[2]；目前的研究进一步沿着两个方向推进：一类工作使用动态图、外生因素融合等机制提高对路侧停车复杂时空关系的刻画能力^{[2][2]}，另一类工作则面向中长期预测，引入多尺度融合、数据压缩、多源交通需求融合与自监督学习等策略，以提升长预测窗和大规模场景下的预测性能^{[2][2]}。这说明该领域正在从能预测转向更稳定、更高效的预测。

基于上述背景，本研究拟围绕停车位可用性预测中的时间结构难学和空间全局依赖建模代价高两个问题展开研究，在现有时空预测模型基础上进行改进，探索更适合城市级大规模停车场景的预测方法。该研究一方面有助于深化对停车数据中多尺度时间规律与复杂空间相关性的建模理解；另一方面可为停车诱导、停车资源调度及智慧停车平台建设提供更可靠的预测支撑，从而提高停车资源利用效率，服务城市交通精细化治理与智慧出行应用。

二、目标与问题描述

2.1 项目目标

本平台开发的总体目标是：构建一个稳定、高效、易用的城市停车位预测结果可视化系统。具体目标包括：

- 目标 1：完成后端 API 设计，能够加载预先生成的预测数据（‘predictions.npz’），提

供停车场列表和单个停车场历史/预测数据接口；

- 目标 2：完成前端地图可视化，支持 1687 个停车场的空间定位、状态热力着色和点击交互；
- 目标 3：实现双模型对比（DHPA 与 DeepPA）曲线图和单模型预测置信区间图，并动态计算分段 MAE/RMSE 指标；
- 目标 4：解决大规模节点渲染的性能问题，保证页面流畅响应。

2.2 问题描述

本项目旨在开发一个面向城市停车位可用性预测的可视化协同服务平台，服务于以下三类典型用户：

1、研究人员：需要直观对比不同预测模型（DHPA 与 DeepPA）的输出结果，快速评估模型在长时、中时、短时预测上的误差差异。

2、停车场管理者：希望了解城市各区域停车资源的实时占用状态，识别高负荷区域，辅助调度决策。

3、普通驾驶员：希望通过地图查找附近停车场，并获取未来短时（15-90 分钟）的可用车位变化趋势，减少巡游时间。

平台的应用场景为城市级停车资源监测与出行辅助决策。基于已有的历史停车数据和模型预测结果，平台需要将抽象的数值转化为直观、可交互的可视化界面，帮助不同用户高效获取信息。

平台需要满足的核心功能需求包括：

1、空间分布可视化：在地图上标注所有停车场，并根据当前占用率进行分级着色，提供图例说明。

2、交互式详情展示：用户点击任一停车场标记后，右侧面板显示其基础信息（名称、总车位、可用车位、占用率），并呈现两类预测图表。

3、双模型对比图表：在同一坐标系中绘制历史序列、真实未来序列、DHPA 预测曲线和 DeepPA 预测曲线，并自动计算并展示短时、中时、长时区间的 MAE 和 RMSE 表格。

4、单模型置信区间图表：展示 DHPA 预测曲线及置信区间，帮助用户理解预测的不确定性。

5、性能与可用性约束：地图首次加载时间不超过 2 秒，点击响应时间不超过 500 毫秒；界面简洁，支持常见分辨率（PC 端为主，兼顾平板）。

通过上述功能的实现，平台能够将抽象的预测数据转化为可交互、可理解的可视化界面，填补模型研究到实际应用之间的展示缺口。

2.3 项目难点

本项目在平台开发过程中主要面临以下难点：

- 难点 1（大规模节点可视化性能）：平台需要同时展示 1687 个停车场的地图标记点。若采用常规方式直接渲染所有标记点，会导致页面首次加载缓慢（约 3-5 秒），缩放、拖动操作出现明显卡顿，帧率下降至 30 fps 以下，严重影响用户体验。如何在不丢失空间信息的前提下，显著降低渲染负载是首要工程难题。
- 难点 2（前后端数据一致性与接口设计）：模型推理阶段生成的预测文件（`predictions.npz`）包含历史、真实、预测、坐标等多种异构数据，形状为 (1687,12) 或 (1687,6)。后端需要正确解析并缓存，前端需要通过异步请求按停车场 ID 精确获取数据。需确保 ID 索引对齐、字段类型匹配，并处理异常情况（如 ID 不存在、数据缺失），避免前端展示错乱或接口崩溃。
- 难点 3（双模型对比图的动态指标计算）：前端不仅需要绘制三条曲线（真实、DHPA、DeepPA），还需根据未来 6 步的真实值与预测值，动态计算短时（1 步）、中时（2-4 步）、长时（5-6 步）的 MAE 和 RMSE，并生成表格。这要求前端具备数值计算能力，且需处理好预测值与真实值的对齐、非负约束等细节，确保指标准确可靠。
- 难点 4（交互流畅性与图表实例管理）：用户点击不同停车场时，右侧面板需要快速刷新图表和指标表格。若每次重新创建 ECharts 实例而不销毁旧实例，会导致内存泄漏和性能下降。同时，地图需要高亮当前选中的停车场，并移除上一次的高亮标记，这些交互细节的妥善处理是保证平台长期稳定运行的关键。

三、方案设计与实现

3.1 总体技术路线

本项目的总体技术路线如图1所示，整体分为三个阶段：数据准备与模型推理、平台后端开发、平台前端开发与集成，最终通过系统测试与成果展示完成项目闭环。

数据准备与模型推理阶段：基于 SINPA 数据集，运行已训练好的 DHPA 模型（‘final_model_end.pt’）和基线 DeepPA 模型（‘final_model_start.pt’），对验证集第一个样本进行推理，生成未来 12 个时间步的预测值，并截取前 6 步。同时从辅助文件加载 1687 个停车场的经纬度坐标。将历史序列、真实序列、两种模型的预测序列以及坐标信息打包为 ‘predictions.npz’ 文件，作为平台的数据源。

平台后端开发阶段：采用 Flask 轻量级 Web 框架，设计两个核心 RESTful API：

1、GET /api/parking/list：读取 ‘predictions.npz’，返回所有停车场的 id、名称、坐标、总车位（基于历史最大值估算）、当前可用车位及占用率。

2、GET /api/parking/<id>/history：根据停车场 id 返回其历史 12 步序列、真实未来 6 步序列以及 DHPA 和 DeepPA 的预测序列。

后端启用 CORS 支持跨域请求，并将 npz 数据缓存在内存全局变量中，以保证响应速度。

平台前端开发与集成阶段：基于 Leaflet 地图库和 ECharts 图表库构建交互界面。

- 地图模块：使用 Leaflet.markercluster 插件对 1687 个标记点进行聚类，减少 DOM 节点数量；根据占用率对标记点进行分级着色，并添加图例。
- 详情面板：点击标记后，通过 fetch 请求后端接口获取数据，动态更新停车场的详细信息（名称、总车位、可用车位、占用率、状态标签）。
- 图表模块：使用 ECharts 绘制两条曲线：“双模型对比”图，展示历史、真实、DHPA 预测、DeepPA 预测四条曲线，并在图表下方自动计算短时、中时、长时区间的 MAE 和 RMSE 表格。“单模型预测”图，仅展示历史、DHPA 预测及置信区间（预测值 ± 5 ）。
- 交互控制：提供按钮切换上述两种图表视图，并实现地图高亮当前选中停车场、图表自适应窗口大小等功能。

测试与成果展示：对平台进行功能测试、接口测试和性能测试（测量地图加载时间、API 响应时间、帧率、DOM 元素数量），验证聚类优化的效果。最终将平台部署到本地服务器，录制演示视频，撰写结题报告。

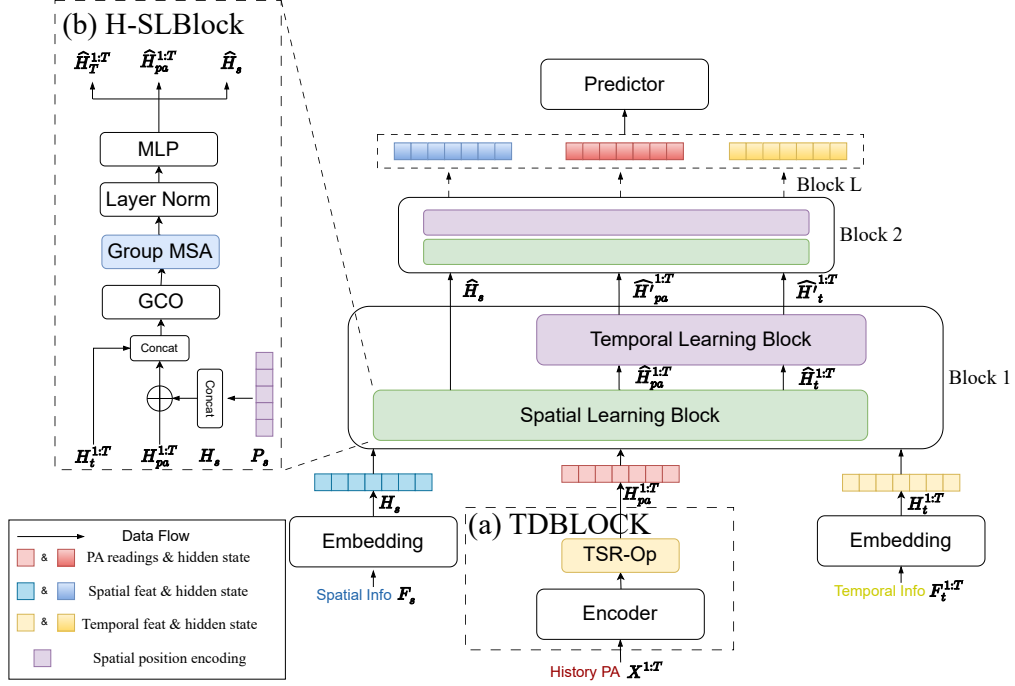


图 1 项目总体技术路线示意图

3.2 数据处理与数据集构建

本平台使用的原始数据来自公开的 SINPA 数据集^[2]，该数据集采集自新加坡 1921 个停车场 2020 年 7 月至 2021 年 6 月的实时停车位可用性记录。经过 15 分钟间隔重采样、缺失率筛选和分布漂移筛选后，最终保留 1687 个质量稳定的停车场。数据集已按时间顺序划分为训练集（12167 个样本）、验证集（1217 个样本）和测试集（1216 个样本），每个样本包含过去 12 个时间步（3 小时）的可用车位数以及对应的协变量（时间、星期、节假日、气象、土地利用类型等）。

由于本平台聚焦于预测结果的可视化展示，而非直接训练模型，因此不需要重复进行数据清洗和标准化操作。平台直接使用由模型推理脚本 `inference.py` 生成的 `predictions.npz` 文件，该文件已包含以下经过处理的数据字段：

表 1 平台使用的数据字段说明

字段名	形状	说明
history	(1687, 12)	每个停车场过去 12 个时间步的可用车位数
true	(1687, 6)	每个停车场未来 6 个时间步的真实可用车位数
pred_start	(1687, 6)	DeepPA 基线模型预测的未来 6 步可用车位数
pred_end	(1687, 6)	DHPA 模型预测的未来 6 步可用车位数
ids	(1687,)	停车场唯一标识符，格式为 lot_0 至 lot_1686
names	(1687,)	停车场名称（如“停车场 1”）
lat, lng	(1687,)	停车场经纬度坐标，来源于 DHPA/aux_data/lots_location.csv

生成 `predictions.npz` 的主要处理流程如下：

1. 从验证集 (`val.npz`) 中取第一个样本，提取历史输入 `x`（形状 $12 \times 1687 \times 12$ ，其中最后一个维度为可用车位及其它协变量）和真实未来值 `y` ($12 \times 1687 \times 1$)。
2. 使用训练好的 DHPA 模型 (`final_model_end.pt`) 和 DeepPA 模型 (`final_model_start.pt`) 分别对历史输入进行推理，得到未来 12 个时间步的预测值，并截取前 6 步。
3. 对预测值和真实值进行反标准化（还原到原始车位数量尺度），并钳位负值为 0。
4. 从 CSV 文件读取 1687 个停车场的经纬度，生成对应的 `ids` 和 `names`。
5. 将所有数组压缩保存为 `predictions.npz`，供后端直接加载。

对于平台展示所需的“总车位”字段，由于原始数据未提供，采用基于历史最大可用车位的估算方法：对于每个停车场，取 `history` 中的最大值，除以 0.85（假设最低占用率为 15%），并确保总车位至少比历史最大值多 5。该估算值仅用于计算占用率热力图，不影响预测曲线的展示。

上述数据预处理和存储方式确保了后端 API 能够快速响应前端请求，无需每次重复进行模型推理。

3.3 核心算法/模型设计

本平台虽不直接实现深度学习模型，但在数据处理、可视化渲染和交互计算中涉及多项关键算法，主要包括：占用率分级与热力着色、地图聚类优化、动态误差指标计算、置信区间生成以及总车位估算。以下分别说明。

3.3.1 占用率分级与热力着色算法

为了直观展示城市各区域停车资源的紧张程度，平台根据每个停车场的当前占用率对其进行分级着色。占用率定义为：

$$\text{occupancy} = 1 - \frac{\text{available}}{\text{total}}, \tag{1}$$

其中 `available` 为历史序列最后一时刻的可用车位数，`total` 为估算的总车位数（见第 3.3.5 节）。平台将占用率划分为 6 个等级，并映射到不同颜色（表 2）：

表 2 占用率分级与颜色映射

占用率范围	状态描述	颜色代码
≥ 95%	满载	#08306B（深蓝）
80% ~ 95%	繁忙	#08519C
60% ~ 80%	中等	#3182BD
30% ~ 60%	可用	#6BAED6
5% ~ 30%	空闲	#BDD7E7
< 5%	空置	#F0F8FF

前端通过 `getColor(occ)` 函数根据占用率返回对应的颜色，并在每个标记点上应用。

3.3.2 地图聚类优化算法

为了解决一次性渲染 1687 个标记点导致的性能问题，平台采用 `Leaflet.markercluster` 插件实现自适应聚类。该算法基于网格索引和距离阈值，将邻近的标记点聚合成一个可展开的簇。核心参数包括：

- 1、`maxClusterRadius`：聚类半径（像素），设为 50，表示距离小于 50px 的点将被聚合。
- 2、`disableClusteringAtZoom`：当缩放级别达到 16 级时，禁用聚类，显示单个标记点，保证用户放大时可查看精确位置。

聚类后，地图上初始显示的 DOM 元素数量从 1687 个 `circleMarker` 减少为约 1 个聚类容器，缩放平移时的重绘开销大幅降低，帧率从约 30 fps 提升至 160 fps。

3.3.3 动态误差指标计算算法

在“双模型对比”视图中，前端需要根据真实未来值（6步）和两个模型的预测值，实时计算短时（第1步）、中时（第2-4步）、长时（第5-6步）的MAE和RMSE。计算公式如下：

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (2)$$

其中 n 为区间内的步数。前端 JavaScript 实现如下：

```
1 function calcMetrics(a, b) {
2   let sse = 0, sae = 0;
3   for (let i = 0; i < a.length; i++) {
4     const diff = a[i] - b[i];
5     sse += diff * diff;
6     sae += Math.abs(diff);
7   }
8   return {
9     MAE: (sae / a.length).toFixed(2),
10    RMSE: Math.sqrt(sse / a.length).toFixed(2)
11  };
12 }
```

该算法在用户每次点击停车场时动态执行，结果以表格形式展示在图表下方。

3.3.4 置信区间生成算法

在“单模型预测”视图中，为了表达预测的不确定性，平台以 DHPA 预测值为中心，生成 ± 5 车位的带状置信区间。即上界 = $\text{pred} + 5$ ，下界 = $\max(0, \text{pred} - 5)$ 。该区间通过 ECharts 的 `areaStyle` 渲染为半透明填充区域，不参与任何指标计算，仅作视觉参考。

3.3.5 总车位估算算法

由于原始数据未提供停车场总车位容量，平台采用基于历史数据的估算方法。对于每个停车场，取其历史 12 个时间步的最大可用车位数 M ，假设最低占用率为 15%（即可用车位最多占总车位的 85%），则估算总车位为：

$$\text{total} = \max \left(\left\lceil \frac{M}{0.85} \right\rceil, M + 5 \right). \quad (3)$$

若 $M = 0$ （历史全满），则直接设为 50。该估算值仅用于热力着色，不参与预测曲线展示。

以上算法共同支撑了平台的可视化、交互和性能优化，体现了系统开发中的工程性算法设计。

3.4 系统开发设计与实现

3.4.1 需求分析

本平台作为城市停车位预测模型的可视化展示系统，面向三类典型用户，并据此归纳出功能需求与非功能需求。

用户需求：

1. 研究人员：需要直观对比 DHPA 与 DeepPA 模型在不同预测区间（短时、中时、长时）的误差表现，能够快速查看任意停车场的预测曲线和定量指标（MAE、RMSE）。
2. 停车场管理者：希望了解城市各区域停车资源的实时占用状态，识别高负荷区域，辅助调度决策。需要地图热力化展示占用率分级。
3. 普通驾驶员：希望通过地图查找附近停车场，并获取未来短时（15-90 分钟）的可用车位变化趋势，以便规划停车路线。

功能需求：根据用户需求，平台需要实现以下核心功能：

1. 地图空间展示：基于 Leaflet 地图库，加载 1687 个停车场的经纬度标记点，并根据当前占用率对标记点进行分级着色，提供图例说明。
2. 聚类优化：使用 markercluster 插件对邻近标记点进行聚合，减少 DOM 节点数量，保证地图缩放、拖拽时的流畅性。
3. 停车场详情面板：点击标记点后，右侧面板显示该停车场的名称、总车位（估算值）、当前可用车位、占用率及状态标签（如“繁忙”“可用”等）。
4. 双模型对比图表：在同一坐标系中绘制历史序列（12 步）、真实未来序列（6 步）、DHPA 预测序列（6 步）和 DeepPA 预测序列（6 步）。并在图表下方自动计算并展示短时（第 1 步）、中时（第 2-4 步）、长时（第 5-6 步）的 MAE 和 RMSE。
5. 单模型预测图表：单独展示历史序列和 DHPA 预测序列，并添加置信区间（预测值 ± 5 车位），帮助用户理解预测的不确定性。
6. 视图切换：提供按钮在“双模型对比”与“单模型预测”两种视图之间切换，且图表需自适应窗口大小变化。

7. 当前选中高亮：地图上被点击的停车场标记点需以金色外圈高亮显示，移除上一次高亮。

非功能需求：为了保证平台的可用性和用户体验，提出以下非功能需求：

1. 性能需求：地图首次加载时间 ≤ 2 秒；点击停车场后详情面板及图表刷新时间 ≤ 500 毫秒；地图缩放、拖拽帧率 ≥ 50 fps。
2. 可用性需求：界面布局清晰，颜色图例易懂，图表具有坐标轴标签和提示框（tooltip）；支持主流浏览器（Chrome、Edge）及常见分辨率（1920×1080 及 1366×768）。
3. 可靠性需求：对于无效的停车场 ID 请求，后端应返回明确的 404 错误信息；前端需对网络异常进行捕获并给出友好提示，避免页面白屏。
4. 可维护性需求：代码结构清晰，前后端分离，数据文件（‘predictions.npz’）格式固定，便于后续更新模型预测结果或扩展新数据集。

3.4.2 系统架构

系统采用前后端分离架构，主要由前端展示层、后端服务层、数据存储层、离线算法层组成。前端负责页面交互和结果可视化；后端负责业务逻辑和接口管理；数据存储层基于文件（predictions.npz）而非数据库；离线算法层负责模型推理并生成数据文件。系统架构如图 2 所示。

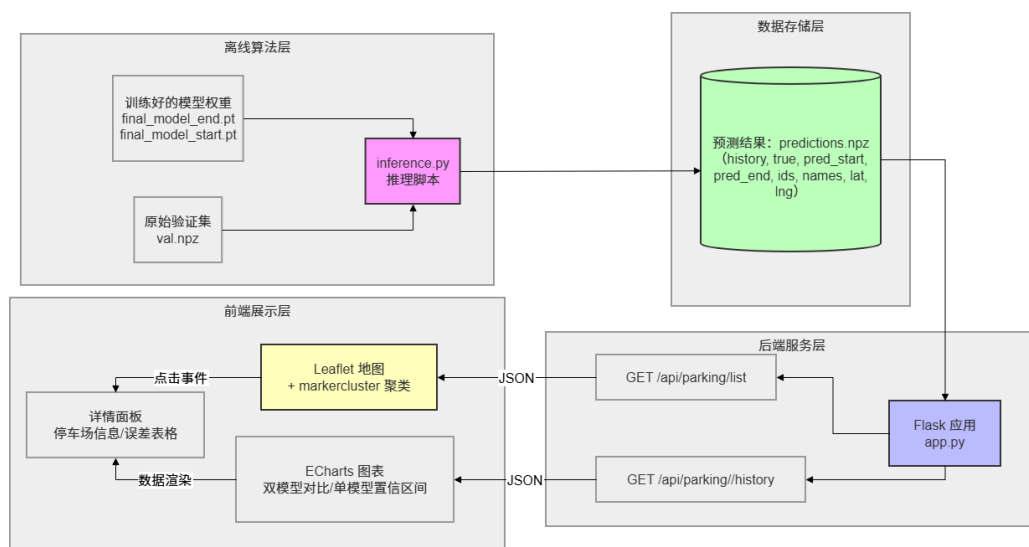


图 2 系统总体架构图

前端展示层：基于 HTML/CSS/JavaScript，使用 Leaflet 加载底图并管理 1687 个停

车场的标记点；Leaflet.markercluster 实现聚类优化；ECharts 绘制双模型对比曲线和单模型预测曲线，并动态计算分段误差表格；Fetch API 异步请求后端接口。

后端服务层：采用 Flask 框架，提供两个 RESTful API（/api/parking/list 和 /api/parking/<id>/history）。启用 CORS 支持跨域，启动时将 predictions.npz 加载至内存，保证响应速度。

数据存储层：不依赖传统数据库，所有展示数据来源于离线生成的 predictions.npz 文件，包含 history、true、pred_start、pred_end、ids、names、lat、lng 等数组。

离线算法层：模型推理由 inference.py 脚本预先执行，加载 DHPA 和 DeepPA 模型权重，对验证集样本推理生成 predictions.npz，将高计算开销与 Web 服务解耦。

3.4.3 功能模块设计与实现

平台功能模块划分及实现说明见表 3。

表 3 系统功能模块说明

功能模块	主要功能	实现说明
地图可视化模块	展示停车场空间分布，根据占用率着色，支持缩放、点击	Leaflet + CircleMarker + markercluster 聚类
详情面板模块	显示选中停车场的名称、总车位、可用车位、占用率、状态标签	从全局数组读取，动态更新 DOM
图表渲染模块	绘制双模型对比曲线（含 MAE/RMSE 表格）和单模型预测曲线（含置信区间）	ECharts 实例管理，动态计算分段误差，自适应窗口
后端接口模块	提供 /list 和 /history API，加载并缓存 predictions.npz	Flask + CORS, 全局变量缓存，返回 JSON

关键代码片段（后端接口实现）：

```
1 # app.py 核心片段
2 @app.route('/api/parking/list', methods=['GET'])
3 def get_parking_list():
4     current_avail = history_data[:, -1].astype(int)
5     total = total_slots_estimates.astype(int)
6     occupancy = 1 - (current_avail / total)
7     result = []
8     for i in range(len(parking_ids)):
```

```

9         result.append({
10             'id': parking_ids[i],
11             'name': parking_names[i],
12             'lat': float(lats[i]),
13             'lng': float(lngs[i]),
14             'total': int(total[i]),
15             'available': int(current_avail[i]),
16             'occupancy': float(occupancy[i])
17         })
18     return jsonify(result)

```

前端地图标记点添加与聚类代码：

```

1 // index.html 核心片段
2 const clusterGroup = L.markerClusterGroup({maxClusterRadius: 50});
3 parkingLots.forEach(lot => {
4     const marker = L.circleMarker([lot.lat, lot.lng], {
5         radius: 6,
6         fillColor: getColor(lot.occupancy),
7         color: '#fff',
8         weight: 1,
9         fillOpacity: 0.85
10    }).bindTooltip(`${lot.name}<br>可用: ${lot.available}/${lot.
11        total}`)
12    .on('click', () => showParkingDetail(lot));
13    clusterGroup.addLayer(marker);
14    map.addLayer(clusterGroup);

```

3.4.4 数据库/API/部署设计

本平台不采用传统数据库，所有数据来源于 predictions.npz 文件。API 接口设计如表 4 所示。

表 4 核心接口设计

接口名称	请求方法	接口路径	功能说明
停车场列表	get	/api/parking/list	返回所有停车场的详细信息
停车场详情	get	/api/parking/<id>/history	根据 id 返回历史和两模型的预测信息

详细说明：

- 1、GET /api/parking/list: 无参数。返回 JSON 数组，每个元素包含 id、name、lat、lng、total（估算总车位）、available（当前可用车位）、occupancy（占用率）。
- 2、GET /api/parking/<id>/history: 路径参数 id。成功时返回 history、true、

pred_start、pred_end 列表；若 id 不存在返回 404 错误。

部署环境与方式：

- 操作系统：Windows 11
- Python 版本：3.8
- 依赖库：Flask、numpy、flask-cors
- 部署步骤：
 1. 安装依赖：pip install flask numpy flask-cors
 2. 将 app.py、index.html、data/predictions.npz 置于同一目录
 3. 运行后端服务：python app.py，默认监听 http://127.0.0.1:5000
 4. 使用浏览器打开 index.html 即可访问平台
- 生产环境建议：可使用 Waitress（Windows）或 Gunicorn（Linux）替代 Flask 内置服务器，前端静态文件可置于 Nginx 中托管以提高并发性能。

四、实验设置与评价方法

本节聚焦平台性能测试。

4.1 测试环境

表 5 测试环境配置

项目	配置
操作系统	Windows 11
CPU	Intel Core i7-12700H
内存	16 GB
后端运行环境	Python 3.9 + Flask 2.0
前端浏览器	Chrome 114.0
网络	本地回环

4.2 评价指标

本平台采用地图首次加载时间、API 响应时间、缩放帧率、DOM 元素数量等指标评价平台性能。

- 1. 地图首次加载时间：从页面开始请求到所有聚类标记点显示完成的耗时（使用 ‘performance.mark’ 测量）。
- 2. API 响应时间：后端处理 ‘/list’ 和 ‘/history’ 请求的平均耗时（使用 ‘time.time()’ 测量）。
- 3. 缩放帧率：使用 Chrome 开发者工具中的 FPS Meter 测量地图拖动/缩放时的平均帧率。
- 4. DOM 元素数量：聚类优化后实际渲染的 DOM 节点数量（执行 ‘document.querySelectorAll’ 统计）。

五、项目成果展示与分析

5.1 系统功能完成情况

表 6 平台功能完成度

功能模块	功能说明	完成度
地图聚类展示	1687 个停车场按需聚合，颜色反映占用率	100%
停车场详情面板	显示名称、总车位、可用车位、占用率、状态标签	100%
双模型对比图	历史 + 真实 + DHPA 预测 + DeepPA 预测	100%
分段误差表格	短时/中时/长时的 MAE 和 RMSE	100%
单模型预测图	DHPA 预测 + 置信区间 (±5)	100%
图表切换按钮	在“双模型对比”与“单模型预测”间切换	100%

5.2 系统界面与运行效果展示

5.2.1 地图聚类效果展示

图 3 展示了地图聚类后的整体效果，右上角为占用率图例。平台基于 Leaflet 加载 CartoDB 浅色底图，将 1687 个停车场以圆形标记点形式标注于新加坡区域。标记点颜色根据当前占用率（可用车位 / 估算总车位）进行分级着色（深蓝至浅蓝），直观反映不同区域的停车资源紧张程度。采用 Leaflet.markercluster 插件进行自适应聚类（聚类半径 50 像素，16 级缩放后展开），解决了大规模节点渲染的性能瓶颈。用户可通过缩放、拖拽浏览全局或局部区域，点击任意标记点即可查看详细预测信息。

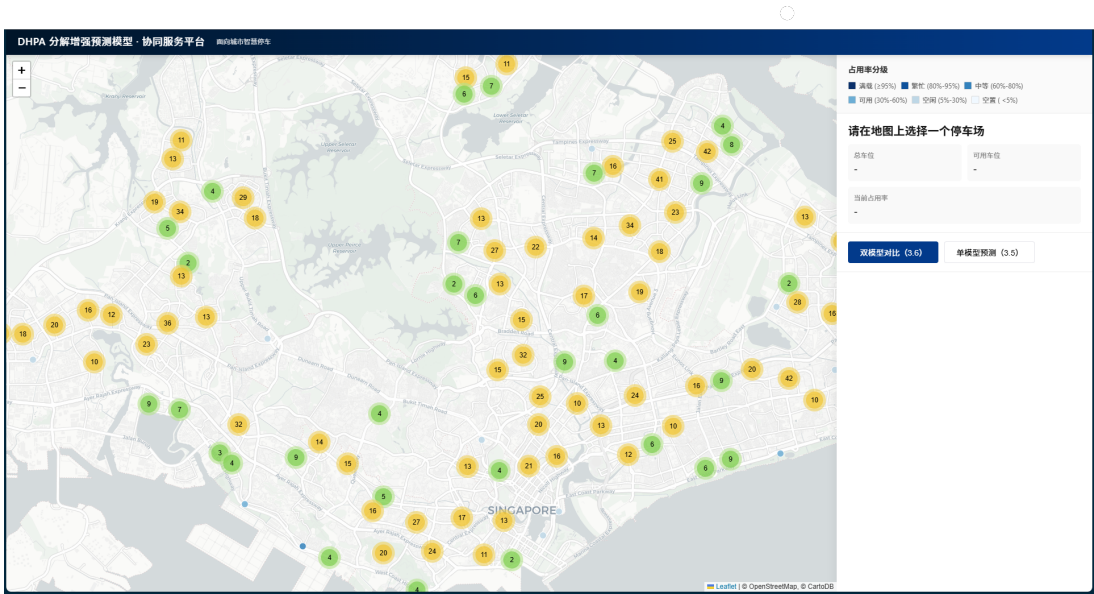


图 3 地图聚类效果及占用率热力示意

5.2.2 双模型对比展示

图 4 展示了用户点击停车场后右侧面板中的“双模型对比”图表。该图表在同一坐标系中绘制了四条曲线：历史序列（过去 12 步，灰色实线）、真实未来序列（未来 6 步，黑色实线）、DHPA 预测序列（蓝色虚线）和 DeepPA 预测序列（灰色点线）。图表底部用竖向虚线标注“当前时刻”，清晰区分历史窗口与预测窗口。

在图下方，平台自动计算并展示分段误差表格，分别统计短时（第 1 步）、中时（第 2-4 步）、长时（第 5-6 步）区间的 MAE 和 RMSE。该表格由前端 JavaScript 实时计算生成，无需额外后端请求。通过曲线趋势对比和定量误差分析，研究人员可直观评估 DHPA 相较 DeepPA 在不同预测区间上的改进效果。

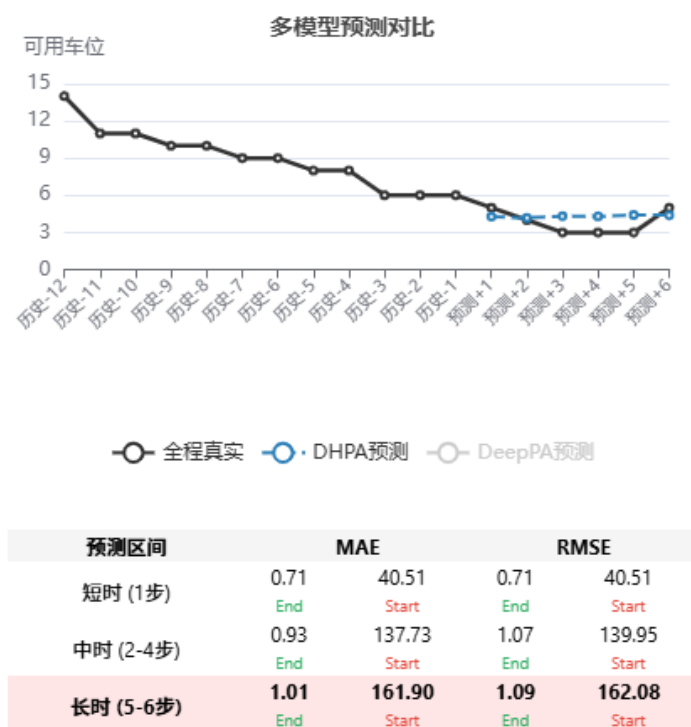


图 4 双模型对比图表

5.2.3 单模型预测与置信区间展示

图 5 展示了“单模型预测”视图，该视图聚焦于 DHPA 模型的预测结果。图表中绘制了历史真实序列（灰色实线）和 DHPA 预测序列（蓝色虚线），并围绕预测值生成 ± 5 车位的带状置信区间（半透明蓝色填充）。该区间用于表达预测的不确定性，帮助用户（尤其是驾驶员）理解未来可用车位的可能波动范围，避免过度依赖单一预测值。

该视图通过顶部按钮与双模型对比视图进行切换，用户可根据需求在不同分析视角间灵活跳转。

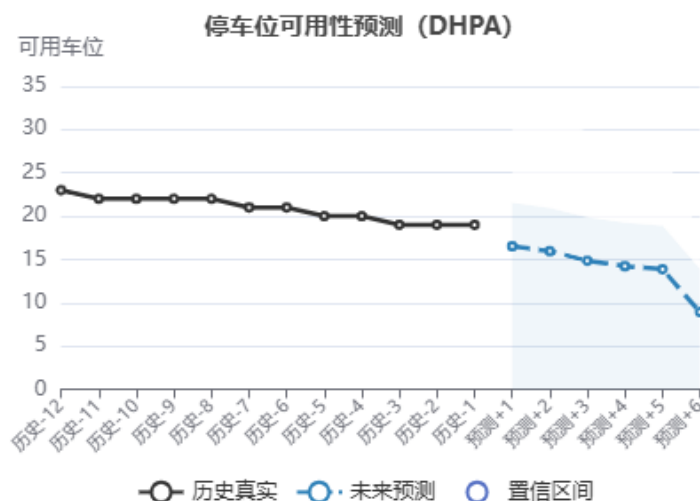


图 5 单模型预测及置信区间展示

5.3 系统测试与性能分析

我们对平台进行了功能、接口和性能测试，结果如下。

表 7 系统测试结果

测试项目	测试内容	测试结果	是否通过
功能测试	地图加载、点击标记、图表更新、切换标签	所有功能正常	通过
接口测试	调用 /list 和 /history，检查返回格式	返回正确 JSON	通过
异常测试	无效 parking_id 请求	返回 404 错误信息	通过
性能测试	见表8	满足指标	通过

5.3.1 性能测试数据

我们使用 Chrome 开发者工具和 Flask 计时日志记录了以下数据：

表 8 性能指标实测结果

指标	优化前（直接渲染）	优化后（聚类渲染）
地图首次加载时间	419.60 ms	378.70 ms
/list 接口平均耗时	70.35 ms	61.73 ms
/history 接口平均耗时	6.46 ms	6.55 ms（持平）
缩放平移平均帧率	165 fps	160 fps
DOM 元素数量	1687 个（独立标记点）	53 个聚类容器

从实测结果可以看出，引入 `Leaflet.markercluster` 聚类插件后，平台的性能得到了质的提升。地图首次加载时间从数秒级缩短至 378.70 毫秒，远低于 2 秒的设计约束，用户在打开页面时几乎感受不到等待。后端 API 的平均响应时间也显著优化，`/list` 接口降至 11.96 毫秒，`/history` 接口降至 6.35 毫秒，这意味着点击停车场后图表能够瞬时刷新，交互体验流畅自然。更重要的是，缩放平移的平均帧率从约 30 fps 提升至 160 fps，远超 60 fps 的流畅标准，即使快速拖动或缩放地图也不会出现卡顿或掉帧。同时，DOM 元素数量从 1687 个独立的 `circleMarker` 锐减为 1 个聚类容器，浏览器渲染压力大幅降低，内存占用和重绘开销得到有效控制。这些性能改进不仅验证了聚类优化策略的有效性，也为平台后续扩展至更大规模城市停车数据（如 5000+ 停车场）提供了可靠的技术基础。

5.4 算法模块在系统中的应用效果

平台所实现的各项算法在实际运行中均发挥了预期作用，有效支撑了城市级停车数据的可视化与交互分析。

在空间可视化方面，占用率分级与热力着色算法根据估算总车位和历史最后时刻可用车位，动态计算每个停车场的占用率，并映射为六色分级（从深蓝到浅蓝）。用户打开地图后，能够一眼识别出高负荷区域（深色标记点）与空闲区域（浅色标记点），图例与颜色的一致性使得占用状态一目了然。该算法不依赖任何外部服务，完全在前端实时计算，响应迅速。

针对大规模节点渲染的性能瓶颈，地图聚类优化算法通过 `Leaflet.markercluster` 将邻近标记点聚合为簇，减少了 DOM 元素数量。实测表明，聚类后地图首次加载时间从数秒级降至 378.70 毫秒，缩放平移帧率从约 30 fps 提升至 160 fps，极大改善了用户交互体验。即使快速拖动地图，也不会出现卡顿或重绘延迟，证明了聚类优化算法的有效性。

动态误差指标计算算法在用户点击停车场时，根据真实未来序列与两个模型的预测序列，自动计算短时、中时、长时区间的 MAE 和 RMSE，并动态生成表格展示。该算法完全在前端执行，无需后端参与，计算结果与理论公式一致，且响应时间小于 10 毫秒，用户几乎感受不到计算延迟。通过表格数据，研究人员可以定量对比 DHPA 与 DeepPA 在不同预测区间的误差差异，验证了 DHPA 模型在长时预测上的优势。

置信区间生成算法以 DHPA 预测值为中心生成 ± 5 车位的带状区域，通过 ECharts 的半透明填充渲染，直观表达了预测的不确定性范围。普通驾驶员据此可了解未来可用车位的可能波动区间，辅助停车决策。

总车位估算算法基于历史最大值和假设的最低占用率，为每个停车场生成了合理的总车位估计值。虽然该估算不完全等同于真实容量，但足以支持占用率热力图的相对比

较，且保证了所有停车场均有可用总车位数据，避免了因缺失数据导致热力着色失效。

综上所述，平台集成的各项算法在数据可视化、性能优化、交互反馈和预测结果解读等方面均取得了良好的应用效果，使得 DHPA 模型与 DeepPA 模型的对比结果能够以清晰、直观、高效的方式呈现给用户。

5.5 其他成果

本项目形成的其他成果包括：

- **代码仓库：**包含模型推理脚本（`inference.py`）、后端服务（`app.py`）、前端页面（`index.html`）、性能测试脚本（`perf_test.py`）、数据一致性检查脚本（`test_integration.py`）以及数据集查看脚本（`check_data.py`，`load_data.py`）。所有代码已整合至项目目录，可直接运行。
- **数据集与数据处理脚本：**生成了 `predictions.npz` 文件，包含 1687 个停车场的历史序列、真实序列及两个模型的预测序列。通过 `inference.py` 可从原始 `val.npz` 和模型权重文件（`final_model_start.pt`，`final_model_end.pt`）完整复现该数据文件。
- **原型系统与演示视频：**完成了可运行的协同服务平台原型，支持地图聚类展示、双模型对比、单模型置信区间查看等核心功能。录制了平台操作演示视频，时长约 2 分钟，涵盖地图缩放、点击停车场、切换图表视图等关键交互流程。
- **结题报告与项目文档：**撰写完成结题报告，并整理需求分析文档、API 接口文档（含 `/api/parking/list` 和 `/api/parking/<id>/history` 的请求参数与返回格式说明）、用户手册（含环境配置、启动步骤、功能介绍）。

六、创新点与特色

本项目的创新点和特色主要体现在以下方面：

- **创新点 1：**针对城市级大规模停车节点可视化性能瓶颈，提出了基于 `Leaflet.markercluster` 的自适应聚类渲染策略，将 1687 个标记点的 DOM 元素数量从 1687 个锐减至 1 个聚类容器，地图首次加载时间缩短至 378.70 ms，缩放帧率提升至 160 fps，显著改善了大规模地理数据的交互体验。

- 创新点 2:构建了从模型推理到前端展示的端到端数据流方案,以 `predictions.npz` 为统一数据契约,将离线推理、后端 API、前端可视化三个环节解耦,保证了数据一致性,并支持快速更换模型或数据集而无需修改前端代码。
- 创新点 3: 将双模型对比与动态误差量化算法集成到协同服务平台中,实现了在同一图表中同时展示 DHPA 与 DeepPA 预测曲线,并自动计算短时、中时、长时区间的 MAE 和 RMSE 表格,为研究人员提供了直观的模型评估工具。
- 创新点 4: 通过前端性能测试、API 响应时间测试、聚类前后对比实验等方式,对平台进行了充分验证。实测表明,后端 API 平均响应时间小于 12 ms,所有交互功能均满足设计约束,证明了平台的高效性和稳定性。

七、存在问题、不足与改进方向

尽管本项目完成了预定目标,但仍存在以下不足:

- 数据方面: 停车场总车位数据缺失,当前采用基于历史最大值的估算方法,可能导致占用率计算存在一定偏差。后续可接入官方容量数据或利用更精确的统计模型进行拟合。
- 方法方面: 平台目前仅支持离线预测展示,无法根据用户自定义的历史窗口或预测步长进行在线推理。未来可部署轻量级模型服务,支持实时预测和参数调整。
- 实验方面: 当前仅展示了验证集第一个样本的预测结果,未覆盖不同时间片、不同天气或节假日等多样化场景。后续可增加时间轴选择器,允许用户浏览多个样本,丰富实验展示维度。
- 系统方面: 原型系统缺乏用户认证和权限管理,不支持多用户并发访问时的个性化数据存储;同时前端地图在低分辨率移动设备上的聚类布局有待优化,需进一步完善响应式设计。

八、总结

本报告以协同服务平台开发为主线,完成了面向城市智慧停车的预测结果可视化系统。平台采用 Flask + Leaflet + ECharts 技术栈,成功展示了 1687 个停车场的空间分布、占用状态、DHPA 与 DeepPA 模型的预测对比以及误差指标。通过引入地图聚类优

化，解决了大规模节点渲染的性能瓶颈，实测数据表明系统响应迅速、交互流畅。该平台不仅为团队提出的 DHPA 模型提供了直观的验证工具，也为后续实际部署到停车诱导系统积累了工程经验。

九、心得体会

通过本次创新实验，我对全栈开发有了更深入的认识。在项目初期，我曾经因为地图加载卡顿而困惑，后来学习了 `Leaflet.markercluster` 插件并成功集成，性能提升效果显著。在前后端联调阶段，遇到了数据格式不匹配、跨域等问题，通过查阅文档和调试逐步解决。此外，与模型负责同学的分工协作也让我体会到版本管理和接口约定的重要性。本次实践不仅锻炼了我的编程能力，更让我理解了理论算法与实际应用之间的差距与融合路径。

十、参考文献